CADO-NFS GPU fork: build options and runtime knobs

Stefan Teleman <stefan.teleman@gmail.com>
in collaboration with Claude Opus 5.5 (Anthropic)

November 10, 2026

Abstract

This document lists every build option, compile-time constant and environment variable that the GPU fork of CADO-NFS (git branch `cado-nfs-gpu`, the commits after upstream `8f0bb2b207`) adds or relies on, and the `cado-nfs.py` and `bwc.pl` parameters it adds or changes. For each one it gives the accepted values, the default, the effect, and the place in the source where it is read.

Contents

1 Introduction **2**
1.1 Who this is for . . . 2
1.2 How to read it . . . 2
1.3 How values are read . . . 2

2 Building **2**
2.1 Requirements . . . 2
2.2 CMake options and configure-time variables . . . 3
2.3 Fixed compiler flags . . . 5
2.4 Example configuration . . . 5
2.5 What a CUDA build adds . . . 5

3 Compile-time macros **5**
3.1 Macros set by the build . . . 6
3.2 Fixed tuning constants (edit the source to change) . . . 6
3.3 Upstream macros that exclude the GPU sieving code . . . 7

4 Environment variables by component **7**
4.1 las: sieving . . . 7
4.2 las: cofactorization . . . 9
4.3 Polynomial selection, stage 1 (collision search) . . . 10
4.4 Polynomial selection: size optimization . . . 10
4.5 Polynomial selection: root optimization (polyselect_ropt) . . . 11
4.6 Linear algebra (bwc, GF(2)) . . . 11
4.7 Characters . . . 12
4.8 Square root . . . 13
4.9 Filtering and compressed files . . . 14
4.10 GPU selection and claiming . . . 14
4.11 cado-nfs.py server, clients and clusters . . . 15
4.12 Testing and simulation . . . 16

5 cado-nfs.py and bwc.pl parameters **16**
5.1 The GPU process layout for sieving 16
5.2 Other cado-nfs.py parameters 17
5.3 bwc.pl parameters 18

6 Differences between the code and its documentation **19**

7 Index **19**

1 Introduction

1.1 Who this is for

This is a reference for people who build the fork from source and run it, on one machine or on a cluster. It assumes familiarity with the CADO-NFS pipeline (polynomial selection, sieving with `las`, filtering, linear algebra with `bwc`, characters, square root) and with CMake.

1.2 How to read it

- **Unset means default.** Nothing in this document needs to be set for a normal run. A CUDA build uses the GPU wherever the fork has GPU code and a GPU is present; every GPU path gives the same results as the CPU path it replaces.
- Each environment variable has an entry with its values, its default, its effect, when to use it, and the source location(s) where it is read (`file:line` at commit `e80379d4b`). Defaults and effects were read from the code at those places.
- Tags: [CUDA build only] means the variable is read only by code compiled in a CUDA build; in a build without CUDA it has no effect. [any build] means it matters in every build. [test/debug only] marks checks, debugging output and simulation knobs that are not meant for production runs. [slow] marks checks that redo the work on the CPU.
- Measured effects are quoted from the fork's `README.gpu.md` files. Unless marked otherwise, they were measured on the development machine: one NVIDIA RTX PRO 4500 Blackwell (sm_120, 32 GB, 82 SMs, 64 MB L2) with an Intel i9-13900K (32 threads). That is not a production configuration. The production targets are the RTX PRO 6000 Blackwell (minimum configuration) and the B200 (optimal configuration). **Every figure given for the RTX PRO 6000 or the B200 is an untested estimate.**

1.3 How values are read

The variables are not all parsed the same way. Table 1 lists the four rules. The safe practice is: use `0` to turn a feature off, `1` to turn it on, and *unset* a check or verbose variable to turn it off (setting it to `0` turns it on).

`las`, `las -t gpu-auto` and `cado-nfs.py` all read `CADO_LAS_GPU_SIEVE` by the integer rule: `0`, `off` and `no` all turn it off.

Most variables are read once per process (cached in a `static`); set them before the program starts.

2 Building

2.1 Requirements

- **CUDA toolkit** with an `nvcc` that supports C++20: the CUDA language standard is set to 20 (`CMakeLists.txt:95`). The fork was built and tested with CUDA 13.2. `find_package(CUDAToolkit REQUIRED)` must succeed (`CMakeLists.txt:97`).

Table 1: Parsing rules for the environment variables.

Rule	Variables
Presence: any value, including `0` and the empty string, turns it on	`CADO_POLYSELECT_GPU_CHECK`, `CADO_ROPT_GPU_CHECK`, `CADO_SQRT_GPU_CHECK`, `CADO_CHARACTERS_GPU_CHECK`, `CADO_POLYSELECT_GPU_VERBOSE`, `CADO_ROPT_GPU_VERBOSE`, `CADO_SQRT_GPU_VERBOSE`, `CADO_BWC_CUDA_VERBOSE`, `CADO_LAS_GPU_SIEVE_NORMS`, `CADO_LAS_GPU_SIEVE_DEBUG`, `SOPT_TEST_NOCPU`
First character: a value starting with `0` turns it off	`CADO_POLYSELECT_GPU`, `CADO_POLYSELECT_GPU_SOPT` (also: starting with `1` forces on), `CADO_ROPT_GPU`, `CADO_ROPT_GPU_ALPHA`, `CADO_SQRT_GPU`, `CADO_CHARACTERS_GPU`, `CADO_BWC_CUDA_RESIDENT`
Integer (C `atoi`): a non-numeric value reads as 0	`CADO_LAS_GPU_SIEVE`, `CADO_LAS_GPU_FILL`, `CADO_LAS_GPU_COFAC` (0 turns off; so `off` or `no` also turn off), `CADO_LAS_GPU_SIEVE_CHECK`, `CADO_GPU_CLAIM_VERBOSE`, `CADO_NO_PIGZ` (non-zero turns on), and every numeric variable (`*_DEVICE`, `*_MIN`, `*_BATCH`, `*_SLAB_LOG`, ...)
Exactly `0`: only the string `0` turns it off; or, for the two marked, any value other than `0` turns it on	`CADO_LAS_GPU_COFILTER`, `CADO_LAS_GPU_FILTER`, `CADO_LAS_GPU_FUSE`, `CADO_LAS_GPU_OVERLAP`; on: `CADO_LAS_GPU_NORMS_ROW`, `CADO_LAS_GPU_SPIN`

- **CMake**: the tree requires 3.18 (`CMakeLists.txt:51`); the default architecture `native` (below) is a CMake 3.24 feature, so with an older CMake on a GPU host give the architectures explicitly. The development machine uses CMake 3.31.
- **CGBN** (NVIDIA's header-only multiple-precision library for CUDA). Required when CUDA is enabled: configuration stops with an error if `cgbn/cgbn.h` is not found (`CMakeLists.txt:101-107`). Used by the cofactor filter of `las` and the sign characters of `characters`.
- **NCCL** (optional): GPU-to-GPU vector exchanges in multi-job `bwc`. Without it, the exchanges go through MPI.
- **MPI** (optional): needed for several `bwc` jobs, and so for several GPUs per `bwc` run with vectors kept on the GPUs, and for linear algebra on several nodes.
- **Run time**: the GPU libraries link the CUDA runtime as a shared library (`CUDA_RUNTIME_LIBRARY Shared` on every GPU target, e.g. `linalg/bwc/CMakeLists.txt:350`), so `libcudart.so` must be found at run time. Optional tools: `zstd` (Section 4.9) and `pigz` (`CADO_NO_PIGZ`).

2.2 CMake options and configure-time variables

`ENABLE_CUDA`

Values: `ON`, `OFF` (CMake option, `-DENABLE_CUDA=OFF`) *Default:* `ON`

Effect. Builds all the GPU code when CMake finds a CUDA compiler (`check_language(CUDA)`). When none is found the build continues without GPU code and prints "CUDA not found; GPU code disabled". When the build has CUDA, `HAVE_CUDA` is defined in `cado_config.h`.

Use. Set `OFF` to build a CPU-only tree on a host that has `nvcc`. With the `./cado-nfs.py`/`make` wrapper, pass it as `CMAKE_EXTRA_ARGS="-DENABLE_CUDA=OFF"` in `local.sh` or the environment (`CMakeLists.txt:66-69`). A plain `ENABLE_CUDA=0` variable in `local.sh` has no effect: the top-level `CMakeLists.txt` imports only a fixed list of environment variables (the `get_override` calls), and `ENABLE_CUDA` is not among them.

Code: `CMakeLists.txt:74-126`, `cado_config_h.in:61`.

`CMAKE_CUDA_COMPILER / CUDACXX`

Values: path to `nvcc` *Default:* `nvcc` found on the `PATH`

Effect. Standard CMake: selects the CUDA compiler. `CUDACXX` is the environment form. The development build uses `/usr/local/cuda/bin/nvcc`.

Use. When `nvcc` is not on the `PATH`, or to choose between several CUDA versions.

`CMAKE_CUDA_ARCHITECTURES / CADO_CUDA_ARCHITECTURES`

Values: a CMake architecture list, e.g. `120`, `100-real;120-real;120-virtual`, `native` *Default:* see below

Effect. Decides which GPUs the binaries contain code for. If `-DCMAKE_CUDA_ARCHITECTURES` is not given, the default is, in order: (1) the environment variable `CADO_CUDA_ARCHITECTURES`, if defined; (2) `native` (the GPUs of the build host), if `nvidia-smi --query-gpu=compute_cap` succeeds and prints a digit; (3) otherwise `100-real;120-real;120-virtual`: B200 (sm_100), RTX PRO 6000 (sm_120), and PTX that later GPUs compile at load time.

A binary that has no code for the GPU it runs on does not fail: each GPU module prints once "this binary has no GPU code for device ...; using the CPU" and takes its CPU path, with the same results (`utils/cado_cuda_check.cuh:18-38`). The exception is `bwc` with `mm_impl=cuda`, which stops, since it has no CPU path at that point (`linalg/bwc/matmul-cuda-kernels.cu:265`).

Use. For a cluster, or whenever the build host is not the run host, give the production list explicitly: `CADO_CUDA_ARCHITECTURES="100-real;120-real;120-virtual"`. A binary built `native` on an RTX PRO 6000 has no code for a B200. The CUDA code builds for sm_100 with CUDA 13.2, but has not run on an sm_100 GPU yet.

Code: `CMakeLists.txt:77-92`.

`CGBN_HOME / CGBN_INCLUDE_DIR`

Values: a directory *Default:* search `$CGBN_HOME`, `$CGBN_HOME/include`, then `/usr/local/include`, `/usr/include`

Effect. `find_path(CGBN_INCLUDE_DIR cgbn/cgbn.h HINTS ENV CGBN_HOME PATH_SUFFIXES include)`. `CGBN_HOME` is an environment variable; `CGBN_INCLUDE_DIR` is the CMake cache variable and can be given with `-D` directly. Not found: configuration fails with "CUDA is enabled but cgbn/cgbn.h was not found".

Use. When CGBN is installed outside the standard include directories. Only CUDA builds look for it.

Code: `CMakeLists.txt:101-108`.

`ENABLE_NCCL`

Values: `ON`, `OFF` *Default:* `ON` (only considered when CUDA is enabled)

Effect. Looks for NCCL. If both the header and the library are found, `HAVE_NCCL` is defined and multi-job `bwc` exchanges vectors GPU to GPU through NCCL; otherwise CMake prints "NCCL not found; multi-GPU bwc exchanges go through MPI". NCCL is used only together with MPI (several `bwc` jobs).

Use. `OFF` to force the MPI transport at build time. At run time `CADO_BWC_GPU_COMM=mpi` does the same without rebuilding.

Code: `CMakeLists.txt:111-123`, `linalg/bwc/CMakeLists.txt:328-331`.

`NCCL_HOME / NCCL_INCLUDE_DIR / NCCL_LIBRARY`

Values: directories / a library path *Default:* system search paths

Effect. `NCCL_HOME` (environment) is a hint for both searches, with suffixes `include` and `lib`, `lib64`. `NCCL_INCLUDE_DIR` and `NCCL_LIBRARY` are the cache variables (development build: `/usr/include`, `/usr/lib64/libnccl.so`).

Code: `CMakeLists.txt:113-116`.

`MPI (environment, at configure time)`

Values: `1`, `yes`, `on`, or a path; `0`, `no`, `off` or empty for none *Default:* unset: no MPI

Effect. Upstream mechanism (`config/mpi.cmake:15`): with `MPI=1` in the environment when CMake runs, `find_package(MPI)` is used and `bwc.pl` gets an `mpiexec`. The fork needs it for one `bwc` job per GPU (vectors on the GPUs, NCCL) and for multi-node linear algebra.

Use. Configure with `MPI=1` and `mpicc` on the `PATH`. The test is on the environment, not on the cache: a later automatic reconfiguration (for example after a `CMakeLists.txt` change) without `MPI` set turns MPI off again. `linalg/bwc/README.gpu.md` notes that with CMake 3.31, `MPI=<path>` makes `find_package` look for a config package and fail with Open MPI.

2.3 Fixed compiler flags

These are set in the CMake files. They are not options; they are listed so that nobody overrides them by accident.

- `-fmad=false` on `sieve/las-sieve-gpu-kernels.cu`, `sieve/las-sieve-gpu-cofilter.cu` (`sieve/CMakeLists.txt:168-169`) and `polyselect/sopt_gpu_kernels.cu` (`polyselect/CMakeLists.txt:27`). It forbids contracting `a*b+c` into a fused multiply-add, so that the lognorms of `las` and the size optimization of `polyselect` do the same double operations as the CPU code, which is built without FP contraction. The bit-identical results depend on it. Do not add `--use_fast_math` or `-fmad=true` to `CMAKE_CUDA_FLAGS`.
- `-lineinfo` on all GPU libraries except the `bwc` kernels: source lines for Nsight Compute; no run-time cost.
- Position-independent code and the shared CUDA runtime on every GPU library.

2.4 Example configuration

A build for the production GPUs on a host without a GPU (a login node), with MPI and NCCL, CGBN in a private directory:

```
export MPI=1
export CGBN_HOME=$HOME/opt/cgbn
export CADO_CUDA_ARCHITECTURES="100-real;120-real;120-virtual"
export CUDACXX=/usr/local/cuda/bin/nvcc
make cmake && make -j32
```

Plain CMake equivalent:

```
MPI=1 cmake -S . -B build/gpu \
    -DCMAKE_CUDA_COMPILER=/usr/local/cuda/bin/nvcc \
    -DCMAKE_CUDA_ARCHITECTURES="100-real;120-real;120-virtual" \
    -DCGBN_INCLUDE_DIR=$HOME/opt/cgbn/include
cmake --build build/gpu -j32
```

Check the configure output for the lines `CGBN: ...`, `CUDA enabled: ... (architectures ...)` and `NCCL enabled: ....`

2.5 What a CUDA build adds

- GPU libraries linked into the existing programs: `las_gpu_kernels` and `facul_gpu_kernels` (`las` and its variants), `polyselect_gpu_kernels`, `sopt_gpu_kernels`, `ropt_gpu_kernels`, `bwc_cuda_kernels` and the `cuda` matmul backend, `characters_gpu`, `sqrt_gpu_mul`.
- `linalg/bwc/cuda_devices`: prints the number of usable CUDA devices. `bwc.pl` and `cado-nfs.py` run it to decide whether to use the GPU (`linalg/bwc/CMakeLists.txt:505-507`).
- Test programs [test/debug only]: `sieve/ecm/facul_gpu_test` (built by default; compares the GPU cofactorization with `facul_all` on a `las -dump-cofactors` file), `polyselect/sopt_gpu_test` and `sqrt/sqrt_gpu_mul_test` (not built by default: `make sopt_gpu_test`, `make sqrt_gpu_mul_test`).

3 Compile-time macros

None of the fork's tuning constants can be overridden with `-D`. Each is a plain `#define` (not guarded by `#ifndef`), so changing one means editing the source. A search of the fork's diff for `#ifndef X` / `#define X` pairs finds only header guards and the upstream macros listed at the end of this section.

3.1 Macros set by the build

HAVE_CUDA, HAVE_NCCL

Values: defined or not *Default:* set by CMake

Effect. Written into `cado_config.h` from `ENABLE_CUDA` / `ENABLE_NCCL` and what CMake found (`cado_config_h.in:61-63`). `HAVE_CUDA` selects every GPU code path; without it the GPU entry points are stubs that return "not used". `HAVE_NCCL` adds the NCCL transport of `bwc` (`linalg/bwc/matmul_top.cpp:250`). Do not define them by hand.

CADO_MATH_AUX_FMA_ASM

Values: defined or not *Default:* derived from the compiler flags

Effect. Defined in `utils/cado_math_aux.hpp:315-319` when compiling for x86-64 with GCC or Clang, with `-mavx` (or higher) but without `-mfma`, and not for the GPU. Then `cado_math_aux::fma` for `double` and `float` issues `vfmadd213sd`/`vfmadd213ss` inline if the CPU has FMA (checked once with `__builtin_cpu_supports("fma")`), instead of calling `std::fma` through the PLT. The result is the same (a fused multiply-add is correctly rounded either way). Measured: c140 polyselect work unit, size optimization 40.2 s → 37.6 s (`polyselect/README.gpu.md`). [any build].

Not a knob: defining it by hand without the conditions above breaks the build (`cpu_has_fma` is declared only inside the condition).

3.2 Fixed tuning constants (edit the source to change)

Constant	Value	File	Meaning
SLAB_LOG	22	sieve/las-sieve-gpu-kernels.cu:53	Base log2 of the positions sieved at a time, best for a 64 MB L2. The run-time value is derived from it (see CADO_LAS_GPU_SLAB_LOG).
SLAB_LOG_MIN, SLAB_LOG_MAX	17, 26	sieve/las-sieve-gpu-kernels.cu:54-55	Clamp for the slab size (whole rows for $\log I \leq 16$, whole tiles).
TILE_LOG	14	sieve/las-sieve-gpu-kernels.cu:46	Positions per small-sieve tile: 32 KB of shared 16-bit sums, 3 blocks per SM (2^{15} allowed one).
SUMMARY_WORDS	4096	sieve/las-sieve-gpu-kernels.cu:562	Words of the survivor-bitmap summary kept in shared memory during resieving (16 KB).
LOGNORM_GUARD_BITS	1	sieve/las-sieve-gpu-kernels.cu:45	Guard bits of the GPU lognorm computation.
LSG_COF_MAXDEG	8	sieve/las-sieve-gpu-kernels.h	Highest polynomial degree the GPU cofactor filter handles; higher degrees leave the filter off.
LFG_BLOCK	128	sieve/las-fill-gpu-kernels.cu	Threads per block of the GPU bucket fill.
FGPU_MAXP, FGPU_MAXC, FGPU_UMAX	8, 4, 120	sieve/ecm/facul-gpu-data.h	Limits of the GPU cofactorization method tables.
CHUNK	4096	linalg/bwc/matmul-cuda-kernels.cu	Maximum indices per work item of the `bwc` product (one warp per item; longer rows are split, with atomic XOR).
MAXSIDES, MAXDEG	4, 8	linalg/characters_gpu.cu	Limits of the GPU character computation.
MAXD, MAXROT, GUARD	8, 7, 0.001	polyselect/sopt_gpu_kernels.cu	Degree limit, rotation degree limit, and the local-descent guard (equal to the CPU's SOPT_LOCAL_DESCENT_GUARD).
NP, LOG2_MAX	3, 40	sqrt/sqrt_gpu_mul.cu	The GPU multiplication uses three primes $c \cdot 2^{40} + 1$.

3.3 Upstream macros that exclude the GPU sieving code

The GPU siever and GPU bucket fill are compiled only when `HAVE_CUDA` is defined and none of `SIQS_SIEVE`, `SUPPORT_LARGE_Q`, `TRACE_K` is (`sieve/las-sieve-gpu.cpp:50`, `sieve/las-fill-gpu.cpp:49`). These upstream macros are set per executable by the existing CMake files (the SIQS siever, the large-q and tracing variants of `las`); those variants use the CPU siever. The GPU cofactorization (`facul_gpu`) is linked into every variant.

4 Environment variables by component

4.1 las: sieving

What runs where in a CUDA build: the GPU siever does the lognorms, the bucket and small sieve, the survivor search and the resieving for a whole special-q, both sides, when there are two sides with factor bases, no sublattices, $I \leq 2^{16}$ and resieving is needed (all default parameter files; `sieve/las-sieve-gpu.cpp:166-176`). When it does not apply, the CPU sieves, and the top-level bucket fill can still run on the GPU.

`CADO_LAS_GPU_SIEVE`

Values: integer; `0` off, non-zero on *Default:* on (1)

[CUDA build only]. *Effect.* Turns the GPU siever on or off in `las`. Off: the CPU siever, with the GPU bucket fill and GPU cofactorization still active if available. Also read by `las -t gpu-auto` (`sieve/las-parallel.cpp:232-235`: off means no GPU, so `auto`) and by `cado-nfs.py` (`scripts/cadofactor/toplevel.py:1128-1133`: off disables the GPU process layout), all three with the integer rule: off when the value's leading integer is 0 or it has none (`0`, `off`, `no`).

Use. To compare with the CPU, or on a host whose GPU is busy with something else. Measured (c120, special-q 1000000–1008000): CPU `-t 32` 8.32 s, GPU siever `-t 32` 3.65 s, GPU siever `-t Machine,8,4` 1.39 s (`sieve/README.gpu.md`).

Code: `sieve/las-sieve-gpu.cpp:157-164`.

`CADO_LAS_GPU_SLAB_LOG`

Values: integer $k > 0$ *Default:* derived: see below

[CUDA build only]. *Effect.* The GPU sieves the region in slabs of 2^k positions. Default: `SLAB_LOG` (22) plus one per doubling of the device's L2 cache over 64 MB (rounded), plus, per special-q, half a power of 2 per doubling of the bucket-sieved factor base entries (both sides) over 426,000 (rounded); clamped to $[17, 26]$. A value set here is used as is (clamped to $[17, 26]$) and turns both adjustments off. Relations do not depend on it.

Measured on the RTX PRO 4500 (Machine,16,2): c120 ($I = 12$) best at 2^{22}; c140 parameters (1.63 million entries) 2^{21} 10.43 s, 2^{22} 7.62 s, 2^{23} 6.75 s, 2^{24} 8.08 s. The RTX PRO 6000 (128 MB L2) and B200 (126 MB) get 2^{23} at c120 by the L2 rule: **an untested estimate**.

Use. To tune the slab size on a GPU it was not measured on.

Code: `sieve/las-sieve-gpu-kernels.cu:972-986`, `sieve/las-sieve-gpu-kernels.cu:1047-1056`.

`CADO_LAS_GPU_FUSE`

Values: `0` turns off *Default:* on

[CUDA build only]. *Effect.* On: one kernel resieves slab s and sieves slab $s + 1$ (`lsg_resieve_sieve`), so the per-lattice work is done once per slab. `0`: separate sieve and resieve passes. Same relations. Measured: c140 5.71 s → 5.63 s, c120 $I = 14$ 6.04 s → 5.59 s.

Use. Debugging or timing comparisons only.

Code: `sieve/las-sieve-gpu-kernels.cu:1115`.

`CADO_LAS_GPU_NORMS_ROW`

Values: set and not `0` turns on *Default:* off

[CUDA build only]. *Effect.* For an algebraic side (degree > 1) whose lognorm approximation has several segments, the default kernel runs one thread per (segment, row) (`lsg_norms_alg`). On: the older one-thread-per-row kernel (`lsg_norms`). Same bytes either way. Measured: that launch 551 → 292 µs, wall time 2.75 → 2.73 s.

Use. Comparisons only.

Code: `sieve/las-sieve-gpu-kernels.cu:1128`.

`CADO_LAS_GPU_SPIN`

Values: set and not `0` turns on *Default:* off (sleeping waits)

[CUDA build only]. *Effect.* Off: a subjob waits for the GPU with an event created with `cudaEventBlockingSync`, so the thread sleeps. On: the wait spins. Measured: spinning was 3/4 of `las`'s CPU time; with sleeping waits `las` used about 1150% CPU instead of about 1700% for the same wall time.

Use. Only to measure the difference; spinning takes CPU that the cofactorization needs.

Code: `sieve/las-sieve-gpu-kernels.cu:965-970`.

`CADO_LAS_GPU_OVERLAP`

Values: `0` turns off *Default:* on

[any build] (has an effect only with the GPU siever). *Effect.* On: a subjob does not wait for the cofactorization tasks of a special-q; it sieves the next special-q in a second `nfs_work` meanwhile. Same relations (order in the output changes). Never used for the descent, batch cofactorization, `-sync` or `-exit-early`. Measured (30000-q c120 range, Machine,16,2): 2.98 s → 2.45 s; full c120 401 → 385 s.

Use. Off only for debugging.

Code: `sieve/las.cpp:1281-1287`.

`CADO_LAS_GPU_FILL`

Values: integer; `0` off *Default:* on (1)

[CUDA build only]. *Effect.* The top-level bucket fill on the GPU, used only on the CPU sieving path (GPU siever off or not applicable, or check mode), when the top level is 1, there are no sublattices, $\log I \leq 16$ and at most 8192 top-level buckets. `0`: the CPU fill.

Use. Comparisons only. The GPU fill has no check mode of its own (an earlier comment named a `CADO_LAS_GPU_FILL_CHECK` that no code read; corrected): compare `las`'s relations with and without it.

Code: `sieve/las-fill-gpu.cpp:71-78`, `sieve/las-fill-gpu.cpp:317-328`.

`CADO_LAS_GPU_SIEVE_CHECK`

Values: integer; non-zero on *Default:* off

[CUDA build only] [test/debug only] [slow]. *Effect.* `las` sieves on the CPU (the GPU siever is not used to produce relations) and also sieves each special-q on the GPU; `process_bucket_region` then compares every region's sieve array with the GPU's at the positions with $\gcd(i, j) = 1$. Up to 10 mismatches are printed; the report line is "GPU sieve check: ... mismatches". Mismatches do not stop the run. Works also with `CADO_LAS_GPU_SIEVE=0`.

Use. After changing the GPU sieve code or the parameters (for example I). Expected: 0 mismatches.

Code: `sieve/las-sieve-gpu.cpp:55-62`, `sieve/las-sieve-gpu.cpp:572-660`.

`CADO_LAS_GPU_SIEVE_NORMS`

Values: presence *Default:* unset

[CUDA build only] [test/debug only]. *Effect.* Only with `CADO_LAS_GPU_SIEVE_CHECK`: the GPU returns its lognorms instead of the final sieve array, and the CPU compares right after its own norm initialization: the lognorms are checked alone.

Code: `sieve/las-process-bucket-region.cpp:981-997`, `sieve/las-sieve-gpu-kernels.cu:1160`.

`CADO_LAS_GPU_SIEVE_DEBUG`

Values: presence *Default:* unset

[CUDA build only] [test/debug only]. *Effect.* Prints the small-sieve entries of the first two special-q per process, and in check mode the small-prime hits of the first two mismatching positions.

Code: `sieve/las-sieve-gpu.cpp:304`, `sieve/las-sieve-gpu.cpp:638`.

4.2 las: cofactorization

`CADO_LAS_GPU_COFAC`

Values: integer; `0` off *Default:* on

[CUDA build only]. *Effect.* The GPU cofactorization (P−1, P+1, ECM chains of the strategy, cofactors below 2^{126}): candidates are queued and run in batches on the GPU; the rest returns to the CPU. Not used for the descent, `-exit-early`, or batch cofactorization. Measured: three large primes, one thread, 222 s → 19 s.

Use. Off to compare with the CPU.

Code: `sieve/ecm/facul-gpu.cpp:196-204`, `sieve/las-process-bucket-region.cpp:908`.

`CADO_LAS_GPU_BATCH`

Values: integer ≥ 1 *Default:* 65536

[CUDA build only]. *Effect.* Candidates (from all subjobs of the process) per GPU cofactorization batch. A batch starts when it is full, after `CADO_LAS_GPU_WAIT`, or when a subjob flushes.

Use. Smaller values lower latency and memory; larger ones fill a larger GPU.

Code: `sieve/las-cofac-gpu.cpp:42,63-64`.

`CADO_LAS_GPU_WAIT`

Values: seconds (floating point) *Default:* 1.0

[CUDA build only]. *Effect.* Longest time a partial batch waits before it is sent. The batch thread wakes every 20 ms, so that is the resolution.

Code: `sieve/las-cofac-gpu.cpp:43,65-66`, `sieve/las-cofac-gpu.cpp:104-111`.

`CADO_LAS_GPU_COFILTER`

Values: `0` turns off *Default:* on

[CUDA build only]. *Effect.* With the GPU siever, the GPU computes each survivor's cofactor (norm in CGBN, minus the 2s, the hit primes, the trial-division primes and the special-q's factors) and applies `check_leftover_norm` (mfb, lpb, gap tests, base-2 strong probable prime test up to 126 bits); rejected survivors are dropped on the GPU. Same relations. `0`: all survivors go to the CPU (the previous behaviour).

Measured on the RTX PRO 4500 (GPU-bound): costs GPU time. c120 $I = 12$, Machine,16,2: wall 2.33 / CPU 28.2 s without, 2.76 / 18.0 s with (before later kernel speed-ups). Full c120: 385 s → 429 s wall, total CPU 3739 → 2797 s. It was designed for the production GPUs, where `las` is expected to be CPU-bound (**untested estimate**).

Use. `0` can be faster on a GPU-bound card such as the RTX PRO 4500; keep the default on RTX PRO 6000 / B200 nodes with few CPU threads per GPU.

Code: `sieve/las-sieve-gpu.cpp:405-410`.

`CADO_LAS_GPU_COFILTER_BITS`

Values: `512`; other values ignored *Default:* chosen per special-q: 256 or 512 bits

[CUDA build only] [test/debug only]. *Effect.* The cofactor filter uses a 256-bit CGBN kernel when the norms fit, else 512 bits, else no filter. `512` forces the wider kernel (if the norms fit 512 bits).

Code: `sieve/las-sieve-gpu.cpp:479-485`.

`CADO_LAS_GPU_FILTER`

Values: `0` turns off *Default:* on

[CUDA build only]. *Effect.* With a strategy file that has an estimate filter (`filter_survivors_by_region`), applies that filter on the GPU (an 8 KB bit table of (S_0, S_1)). `0`: the filter is not applied on the GPU. No effect without such a strategy file.

Code: `sieve/las-sieve-gpu.cpp:499`.

4.3 Polynomial selection, stage 1 (collision search)

CADO_POLYSELECT_GPU

Values: first character 0 turns off *Default:* on

[CUDA build only]. *Effect.* Turns off *all* of polyselect's GPU code: the collision search (collision_on_sq_gpu: the GPU flags the special-q roots with a collision, the CPU then redoes those) and the GPU size optimization. Output identical either way. It does *not* affect polyselect_ropt (see CADO_ROPT_GPU). Used when $p^2 < 2^{62}$, $M < 2^{61}$ and the values fit 8192 slices (all parameter files).

Measured (work units, CPU vs GPU): c120 2 ad -t 1 46.0 vs 19.3 s; c140 75.3 vs 21.8 s; c160 208.3 vs 26.5 s.

Code: polyselect/polyselect_collisions.cpp:1172-1178, polyselect/sopt_gpu.cpp:45-48.

CADO_POLYSELECT_GPU_VERBOSE

Values: presence *Default:* unset

[CUDA build only]. *Effect.* At exit, prints the number of special-q roots tested on the GPU, the GPU time, the number passed to the CPU and the bin overflows; with the GPU size optimization, its descent counts.

Code: polyselect/polyselect_collisions.cpp:1183, polyselect/sopt_gpu.cpp:27.

CADO_POLYSELECT_GPU_CHECK

Values: presence *Default:* unset

[CUDA build only] [test/debug only] [slow]. *Effect.* For every special-q root, computes the exact answer (all values sorted on one CPU thread) and the CPU's decisional pass, and compares them with the GPU's flag; a collision the GPU missed aborts. Use a small -nq for large P.

Code: polyselect/polyselect_collisions.cpp:1301.

CADO_POLYSELECT_GPU_DEVICE

Values: integer device number (taken modulo the device count) *Default:* local rank, else a claimed GPU

[CUDA build only]. *Effect.* The GPU of polyselect (collisions, size optimization) and of polyselect_ropt. See Section 4.10.

Code: polyselect/polyselect_gpu_kernels.cu:267-272, polyselect/sopt_gpu_kernels.cu:668-671, polyselect/ropt_gpu_kernels.cu:112-115.

4.4 Polynomial selection: size optimization

CADO_POLYSELECT_GPU_SOPT

Values: first character 0: CPU; 1: GPU *Default:* GPU only if the device's FP32/FP64 throughput ratio is at most 4

[CUDA build only]. *Effect.* Runs the local descents of the size optimization on the GPU, one per thread, with the CPU's double operations (results identical; a comparison too close to call hands the descent back to the CPU). The default reads cudaDevAttrSingleToDoublePrecisionPerfRatio: ratio ≤ 4 (H100, B200 class) uses the GPU, larger ratios (RTX PRO 4500 and RTX PRO 6000: 64) the CPU. The CPU path is also used whenever polyselect runs with verbosity above 1.

Measured on the RTX PRO 4500: one process alone gains (c120 work unit -t 1: 21.1 $\rightarrow$ 8.1 s), but 16 clients sharing the GPU took 57.9 s against 39.7 s on the CPU: hence the default. On a B200 the GPU path is the default; its speed there is **untested**.

Use. 1 to measure the GPU path on a new card; 0 to force the CPU.

Code: polyselect/sopt_gpu.cpp:39-54, polyselect/sopt_gpu_kernels.cu:763-770, polyselect/size_optimization.cpp:1141.

SOPT_TEST_NOCPU

Values: presence *Default:* unset

[CUDA build only] [test/debug only]. *Effect.* For sopt_gpu_test only: compare at most the first 2000 descents with the CPU instead of all.

Code: polyselect/sopt_gpu_test.cpp:185.

4.5 Polynomial selection: root optimization (polyselect_ropt)

`CADO_ROPT_GPU`

Values: first character `0` turns off *Default:* on

[CUDA build only]. *Effect.* The root sieve arrays (lines recorded on the CPU, applied on the GPU; the int16 array is the CPU's) and the root counts of `get_alpha` on the GPU. `0`: both on the CPU. Measured (one c120 polynomial, one P-core): CPU 6.9 s, GPU sieve 4.2 s, GPU sieve and root counts 2.6 s; the c120's 48 polynomials 64 → 23 s.

Code: `polyselect/ropt_gpu.cpp:76-90`.

`CADO_ROPT_GPU_ALPHA`

Values: first character `0` turns off *Default:* on

[CUDA build only]. *Effect.* Only the GPU root counts for `get_alpha`; the GPU sieve stays. Also off when `CADO_ROPT_GPU=0`, when the alpha bound is ≥ 65536 or a degree is outside 1–31.

Code: `polyselect/ropt_gpu.cpp:117-131`.

`CADO_ROPT_GPU_MIN`

Values: integer (entries) *Default:* 16384 (2^{14})

[CUDA build only]. *Effect.* Sieve arrays shorter than this stay on the CPU (the GPU has a fixed cost per call). Arrays longer than 2^{26} always stay on the CPU (memory).

Code: `polyselect/ropt_gpu.cpp:51-58`, `polyselect/ropt_gpu.cpp:92-102`.

`CADO_ROPT_GPU_CHECK`

Values: presence *Default:* unset

[CUDA build only] [test/debug only] [slow]. *Effect.* Sieves every array on the CPU too and compares; recounts every root count with `mpz_poly_roots_ulong`. Any difference aborts.

Code: `polyselect/ropt_gpu.cpp:168`, `polyselect/ropt_gpu.cpp:208`.

`CADO_ROPT_GPU_VERBOSE`

Values: presence *Default:* unset

[CUDA build only]. *Effect.* At exit: sieve arrays, lines and GPU time.

Code: `polyselect/ropt_gpu.cpp:38`.

`CADO_ROPT_SIEVE_BLOCK`

Values: integer > 0 (entries) *Default:* `L1_cachesize` = 16384

[any build] [test/debug only]. *Effect.* Block size of the CPU root sieve. The blocks only keep the array in the L1 cache; the result does not depend on them (the fork fixed two bugs where it did). Arrays recorded for the GPU use one block per row regardless.

Use. To check that the result does not depend on the block size.

Code: `polyselect/ropt_stage2.cpp:652-664`.

4.6 Linear algebra (bwc, GF(2))

`CADO_BWC_CUDA_DEVICE`

Values: integer (modulo the device count); empty means unset *Default:* local rank plus a per-process counter

[CUDA build only]. *Effect.* The GPU of each matrix block of `mm_impl=cuda`. Default: *local_rank* + k·*local_size* modulo the device count, where k counts the blocks of this process, *local_rank* is the first set of `OMPI_COMM_WORLD_LOCAL_RANK`, `MPI_LOCALRANKID`, `SLURM_LOCALID` (else 0) and *local_size* the first set of `OMPI_COMM_WORLD_LOCAL_SIZE`, `MPI_LOCALNRANKS`, `SLURM_NTASKS_PER_NODE` (else 1). `bwc` does not use the lock-file claim.

Code: `linalg/bwc/matmul-cuda-kernels.cu:76-100`.

`CADO_BWC_CUDA_RESIDENT`

Values: first character `0` turns off *Default:* on

[CUDA build only]. *Effect.* On: `krylov`, `mksol` and `secure` keep their vectors on the GPU for a whole interval; the host sees them only at checkpoints. Applies only with one thread per job, no interleaving and one matrix per job (what `bwc.pl` sets up for the GPU). `0`: vectors on the host, copied around each product.

Code: `linalg/bwc/matmul_top.cpp:312-320`.

`CADO_BWC_GPU_COMM`

Values: `nccl`, `mpi` *Default:* NCCL if built in, else MPI

[CUDA build only]. *Effect.* Transport of the vector exchanges between `bwc` jobs (several MPI jobs, resident vectors). Any other value, or `nccl` in a build without NCCL, prints "CADO_BWC_GPU_COMM=... is not available; vectors stay on the host".

Use. `mpi` if NCCL fails on a network; NCCL needs a network it can use (see `NCCL_SOCKET_IFNAME`).

Code: `linalg/bwc/matmul_top.cpp:340-356`.

`CADO_BWC_CUDA_VERBOSE`

Values: presence *Default:* unset

[CUDA build only]. *Effect.* Each process prints at the end: device, number of products, kernel and copy time.

Code: `linalg/bwc/matmul-cuda.cpp:92`.

`CADO_BWC_GPUS_PER_NODE`

Values: non-negative integer *Default:* the local node's GPU count

[CUDA build only] (read by `bwc.pl`). *Effect.* Number of GPUs of each node for the multi-node layout of `bwc.pl` (Section 5.3). If the local node has no GPU, `bwc.pl` also assumes this many and uses `mm_impl=cuda`. Ignored with `gpu=0`, a prime other than 2 or an explicit non-`cuda` `mm_impl`.

Use. When `bwc.pl` (that is, the `cado-nfs.py` server) runs on a node without GPUs, or with a different number than the compute nodes.

Code: `linalg/bwc/bwc.pl:241-244`.

`SLURM_JOB_NUM_NODES`

Values: integer *Default:* (set by Slurm)

Effect. Read by `bwc.pl` when neither `hosts=` nor `hostfile=` is given: the number of nodes for the multi-node GPU layout.

Code: `linalg/bwc/bwc.pl:235-237`.

`CADO_BWC_SIMULATE_GPUS`

Values: integer $n > 0$ *Default:* unset

[CUDA build only] [test/debug only]. *Effect.* `cuda_devices` reports n devices when there is at least one real one; the blocks then share the real GPUs (device choice modulo the real count), and `bwc` sets `NCCL_MULTI_RANK_GPU_ENABLE=1` (if not set) so that NCCL accepts several ranks per GPU. Exercises the multi-GPU code on one card, with no speed-up. Because `cado-nfs.py` also runs `cuda_devices` to count the GPUs for `las`, this variable changes `cado-nfs.py`'s sieving layout too. Pass it to MPI ranks with `mpi_extra_args="-x CADO_BWC_SIMULATE_GPUS"`.

Code: `linalg/bwc/cuda_devices.cpp:18-20`, `linalg/bwc/matmul_top.cpp:262-265`.

Measured, c120 matrix (635,213 rows): `bwc` total 276 s on the CPU (bucket, 32 threads) against 40.3 s on the GPU; c140 matrix: 3.34–3.44 ms per 128-bit product (`linalg/bwc/README.gpu.md`).

4.7 Characters

`CADO_CHARACTERS_GPU`

Values: first character `0` turns off *Default:* on

[CUDA build only]. *Effect.* The character matrix on the GPU: the pairs are read first, then the GPU computes the Legendre symbols (64-bit arithmetic) and the signs of $F(a, b)$ (CGBN, 256 to 2048 bits). Same bits as the CPU. Measured (c120): CPU time 16.4 → 2.9 s, wall 2.3 → 2.1 s.

Code: `linalg/characters.cpp:387-391`.

`CADO_CHARACTERS_GPU_CHECK`

Values: presence *Default:* unset

[CUDA build only] [test/debug only] [slow]. *Effect.* Recomputes every row with `eval_64chars` on the CPU, prints "characters GPU check: N words differ" and aborts if $N \neq 0$.

Code: `linalg/characters.cpp:450-471`.

`CADO_CHARACTERS_GPU_DEVICE`

Values: integer (modulo the device count) *Default:* local rank, else a claimed GPU

[CUDA build only]. *Effect.* The GPU of `characters`. Section 4.10.

Code: `linalg/characters_gpu.cu:157-172`.

4.8 Square root

`CADO_SQRT_GPU`

Values: first character `0` turns off *Default:* on

[CUDA build only]. *Effect.* Products of large integers in `sqrt` (product trees and Newton lifting) go to a GPU NTT multiplication (three primes, CRT on the GPU); the product is the same integer as `mpz_mul`'s. `0`: GMP only. Measured: 2^{18}-limb product 68 ms (GMP) vs 4.7 ms; c120 square root step 27.9 → 13.1 s (with 4 dependencies at once).

Code: `sqrt/sqrt_gpu.cpp:48-60`.

`CADO_SQRT_GPU_MIN`

Values: integer (64-bit limbs) *Default:* 32768 (2^{21} bits)

[CUDA build only]. *Effect.* A product goes to the GPU only if *both* operands have at least this many limbs. 8192 to 65536 gave the same times on the c120.

Code: `sqrt/sqrt_gpu.cpp:57-58`, `utils/gmp_mul_hook.hpp:20-26`.

`CADO_SQRT_GPU_CHECK`

Values: presence *Default:* unset

[CUDA build only] [test/debug only] [slow]. *Effect.* Every GPU product is also computed with `mpz_mul`; a difference aborts.

Code: `sqrt/sqrt_gpu.cpp:30-41`.

`CADO_SQRT_GPU_VERBOSE`

Values: presence *Default:* unset

[CUDA build only]. *Effect.* At the end: number of GPU products, threshold, time, number of GPUs used.

Code: `sqrt/sqrt_gpu.cpp:62-72`.

`CADO_SQRT_GPU_DEVICE`

Values: integer (modulo the device count) *Default:* all visible GPUs

[CUDA build only]. *Effect.* Default: every visible GPU is used; each thread of `sqrt` (one dependency) takes the next GPU in turn. A value here (or an MPI/Slurm local rank) restricts `sqrt` to that one GPU. `sqrt` does not use the lock-file claim.

Code: `sqrt/sqrt_gpu_mul.cu:276-296`.

4.9 Filtering and compressed files

`CADO_FILTER_PARSE_THREADS`

Values: integer ≥ 1 *Default:* 4 if the machine has at least 8 hardware threads, else 1

[any build]. *Effect.* Number of threads that parse relations in `filter_rels`, which reads the relation files of `dup1`, `dup2`, `purge`, `merge`, `replay`, `characters`, `sqrt` and others. 1: the old way (parsing in the reader thread, which was the slowest thread of `dup2` and `purge`).

Code: `utils/filter_io.cpp:66-74`, `utils/filter_io.hpp:2039-2049`.

`CADO_NO_PIGZ`

Values: integer; non-zero on *Default:* off

[any build]. *Effect.* By default, `.gz` files are read with `pigz -dc` and written with `pigz -c --fast` when `pigz` is found on the `PATH` (same format; compresses on all cores, reads about twice as fast as `gzip`). Non-zero: use the upstream `gzip` commands. Applies to every program that opens files through `fopen_maybe_compressed` or `fstream_maybe_compressed`.

Code: `utils/gzip.cpp:329-355`.

The fork also writes `.zst` files with `zstd -1 -q` (upstream: `zstd -3`; `utils/gzip.h:146`). Which suffix `cado-nfs.py` uses is set by `tasks.compression` (Section 5).

4.10 GPU selection and claiming

Every GPU program picks its device as in Table 3. A number given in a variable is taken modulo the number of visible devices. `CUDA_VISIBLE_DEVICES` restricts the devices every program sees (CUDA's own variable; device numbers then count the visible ones).

Table 3: Device selection, in order of precedence.

Program	Variable	Local rank variables	Otherwise
`las` (siever, bucket fill, cofactorization)	`CADO_LAS_GPU_DEVICE`	`OMPI_COMM_WORLD_LOCAL_RANK`, `MPI_LOCALRANKID`, `SLURM_LOCALID`	claim, tag `las`; decided once at start-up
`polyselect`, `polyselect_ropt`	`CADO_POLYSELECT_GPU_DEVICE`	`OMPI_COMM_WORLD_LOCAL_RANK`, `SLURM_LOCALID`	claim, tag `polyselect`
`characters`	`CADO_CHARACTERS_GPU_DEVICE`	`OMPI_COMM_WORLD_LOCAL_RANK`, `SLURM_LOCALID`	claim, tag `characters`
`sqrt`	`CADO_SQRT_GPU_DEVICE`	`OMPI_COMM_WORLD_LOCAL_RANK`, `SLURM_LOCALID`	all GPUs, threads in turn
`bwc`	`CADO_BWC_CUDA_DEVICE`	`OMPI_COMM_WORLD_LOCAL_RANK`, `MPI_LOCALRANKID`, `SLURM_LOCALID`	rank + k·size (see `CADO_BWC_CUDA_DEVICE`)

The claim (`utils/gpu_claim.cpp`). With more than one device (or `CADO_GPU_SIMULATE_NDEV` > 1), a process takes lock files `<dir>/<tag>.<dev>.<slot>.lock` with `flock`, where `<dir>` is `$TMPDIR/cado-gpu-<uid>` (or `/tmp/cado-gpu-<uid>`), trying slot 0 of every device first, then slot 1, and so on up to 256 slots. The lock lasts as long as the process. If the directory cannot be created, or every slot is taken, the device is the process id modulo the device count. All GPU contexts of one process with the same tag get the same answer, so `polyselect`'s collision search and size optimization share one GPU. Different tags do not see each other's locks.

las. `las` decides its GPU once, at start-up (`choose_gpu_device`, `sieve/las.cpp:1231-1244`): `CADO_LAS_GPU_DEVICE` if set; else the first set of `OMPI_COMM_WORLD_LOCAL_RANK`, `MPI_LOCALRANKID`, `SLURM_LOCALID`; else the claim with tag `las`; modulo the device count. It writes the result into `CADO_LAS_GPU_DEVICE` in its own environment, and the siever, the bucket fill and the cofactorization all read that variable first, so they use the same GPU. With one GPU the claim returns 0 at once (it matters there only with `CADO_GPU_SIMULATE_NDEV`: 5 `las` on 4 simulated GPUs took 0, 1, 2, 3, 0). History: before commit `e80379d4b`, `las` used a lock of its own, `/tmp/cado-las-gpu-<uid>-<dev>.lock`, on multi-GPU nodes only, and its parts read the local-rank variables differently.

MPI_LOCALRANKID. `polyselect`, `polyselect_ropt`, `characters` and `sqrt` do not read `MPI_LOCALRANKID` (MPICH, Intel MPI); `las` and `bwc` do. Under such an MPI, give the `*_DEVICE` variable explicitly for those programs if the device matters.

`CADO_LAS_GPU_DEVICE`

Values: integer (modulo the device count) *Default:* see above

[CUDA build only]. *Effect.* The GPU of the `las` siever, bucket fill and cofactorization.

Use. To pin a `las` process to a GPU, for example when running several by hand.

Code: `sieve/las-sieve-gpu-kernels.cu:958-961`, `sieve/las-fill-gpu-kernels.cu:197-200`, `sieve/ecm/facul-gpu-kernels.cu:1061-1071`, `sieve/las.cpp:1231-1244`.

`CADO_GPU_CLAIM_VERBOSE`

Values: integer; non-zero on *Default:* off

[CUDA build only]. *Effect.* Prints each lock-file claim: "# *tag*: GPU *d* of *n* (pid ...)". Only when a claim is made (more than one device, or simulated).

Code: `utils/gpu_claim.cpp:56-59`.

`TMPDIR`

Values: a directory *Default:* `/tmp`

Effect. Standard variable; the claim's lock directory is `$TMPDIR/cado-gpu-<uid>` (mode 0700), for every program that claims, `las` included.

Code: `utils/gpu_claim.cpp:31-33`.

`OMPI_COMM_WORLD_LOCAL_RANK, MPI_LOCALRANKID, SLURM_LOCALID`

Values: integer *Default:* (set by the MPI launcher or Slurm)

Effect. Standard variables. When one is set (see Table 3 for which program reads which), it chooses the device modulo the device count instead of the claim. Note that any program started under `srun` or `mpirun` has one set, so several *different* programs on a node under one launcher may pick the same device.

4.11 cado-nfs.py server, clients and clusters

`CADO_LAS_NGPU`

Values: non-negative integer (digits only) *Default:* the server's GPU count (`cuda_devices`)

[CUDA build only] (read by `cado-nfs.py`). *Effect.* The number of GPUs per sieving node that the GPU process layout assumes (Section 5.1). `0` turns the layout off without turning off the GPU siever.

Use. When the `cado-nfs.py` server has no GPU, or another number than the sieving nodes. Set `-t` to a sieving node's threads too. For nodes of different kinds use `tasks.sieve.las.threads=gpu-auto`.

Code: `scripts/cadofactor/toplevel.py:1124-1150`.

`CADO_SIEVE_WU_LIMIT`

Values: integer; `0` or unset: no limit *Default:* set by `cado-nfs.py` to the GPU count when the GPU layout applies

[any build] (Python). *Effect.* Read by the server's workunit database: a sieving workunit (`_sieving_` in its name) is handed out only while fewer than the limit are assigned. With a client that sends its host name, the count is per host and the limit is $max(1,$ the GPU count the client sent); an older client counts against the global limit. While it is set, the filtering also asks for at least 5% more relations per round (`add_ratio` ≥ 0.05).

Use. Normally not set by hand: `cado-nfs.py` sets it (overwriting any value) when the GPU layout applies. Setting it by hand applies the same per-host limit and filtering request with a custom layout.

Code: `scripts/cadofactor/toplevel.py:1072`, `scripts/cadofactor/wudb.py:386-428`, `scripts/cadofactor/cadotask.py:4967-4969`.

`CADO_CLIENT_NGPU`

Values: non-negative integer (digits only) *Default:* `nvidia-smi -L` count, capped by `CUDA_VISIBLE_DEVICES`

[any build] (Python, `cado-nfs-client.py`). *Effect.* The GPU count the client sends with each workunit request. Default: the lines of `nvidia-smi -L` starting with `GPU`, at most the number of entries of `CUDA_VISIBLE_DEVICES` if that is set; 0 without `nvidia-smi`. The server then gives the host at most $max(1, n)$ sieving workunits at a time (only when `CADO_SIEVE_WU_LIMIT` is active on the server). A node reporting 0 GPUs still gets one.

Use. To override detection, or [test/debug only] to make clients on one machine pose as nodes with other GPU counts.

Code: `cado-nfs-client.py:1823-1847`, `scripts/cadofactor/api_server.py:526-537`.

`CADO_CLIENT_HOST`

Values: a string *Default:* `socket.gethostname()`

[any build] (Python) [test/debug only]. *Effect.* The host name the client sends; the per-host limit counts clients by it.

Use. Testing several "nodes" on one machine.

Code: `cado-nfs-client.py:1812-1817`.

`NCCL_SOCKET_IFNAME (and NCCL's other variables)`

Values: interface names *Default:* NCCL's choice

Effect. NCCL's own variable, not read by the fork: which network interfaces NCCL uses between nodes. Relevant to multi-node `bwc`. Pass it to the ranks with `mpi_extra_args="-x NCCL_SOCKET_IFNAME"` (a `bwc.pl` parameter).

4.12 Testing and simulation

`CADO_GPU_SIMULATE_NDEV`

Values: integer $n > 0$ *Default:* unset

[CUDA build only] [test/debug only]. *Effect.* The lock-file claim chooses among n devices instead of the real count (the result is then taken modulo the real count; this includes `las`'s choice at start-up), and `sqrt` uses n logical GPUs, each with its own tables, on the real ones. It does not affect `las -t gpu-auto`, `bwc` (see `CADO_BWC_SIMULATE_GPUS`) or `cado-nfs.py`.

Code: `utils/gpu_claim.cpp:24-26`, `sqrt/sqrt_gpu_mul.cu:286-290`.

Other test-only variables, described above: `CADO_BWC_SIMULATE_GPUS`, `CADO_CLIENT_HOST`, `CADO_CLIENT_NGPU` (when used to fake nodes), `SOPT_TEST_NOCPU`, `CADO_LAS_GPU_COFILTER_BITS`, and the `*_CHECK`, `*_DEBUG` and `CADO_LAS_GPU_SIEVE_NORMS` variables.

5 cado-nfs.py and bwc.pl parameters

5.1 The GPU process layout for sieving

`las` is fastest as one process per GPU with many 2-thread subjobs (each subjob sieves a special-q on the GPU while its threads cofactor the previous one). `cado-nfs.py` sets this up by itself (`scripts/cadofactor/toplevel.py:1039-1090`) when:

- the GPU count $g > 0$: `CADO_LAS_NGPU` if set, else the output of `lib/linalg/bwc/cuda_devices`; 0 if `CADO_LAS_GPU_SIEVE` is off (integer rule);
- and either `tasks.sieve.las.threads=gpu-auto` is given, or all of: `tasks.sieve.las.threads` not given, no `--client-threads`, and the default parameter file is used (`N` on the command line, no parameter file given).

It then:

1. sets `tasks.sieve.las.threads=Machine,`J`,2` with $J = \max(1, \lfloor \max(4, \lfloor T/g \rfloor)/2 \rfloor)$, $T =$ `tasks.threads` (unless `gpu-auto`). Examples: $T = 32$, $g = 1$: `Machine,16,2`; $T = 32$, $g = 8$: `Machine,2,2`;
2. sets `CADO_SIEVE_WU_LIMIT`$= g$ in its environment: at most g sieving workunits at a time per node;
3. multiplies `tasks.sieve.qrange` by 5, unless an option ending in `qrange` is on the command line (each `las` pays about 0.3 s to start; c120 sieving 7.5 → 5.4 min);
4. sets `slaves.nrclients` to $\lceil T/P \rceil$, $P =$ `tasks.polyselect.threads` (if unset), since the sieving uses one client per GPU and the other clients serve the other tasks (`scripts/cadofactor/toplevel.py:1216-1221`).

It logs "g GPU(s) for las: tasks.sieve.las.threads=...". Setting `tasks.sieve.las.threads` to anything other than `gpu-auto`, or `CADO_LAS_GPU_SIEVE=0` or `CADO_LAS_NGPU=0`, turns it off. Measured c120 layouts (32 threads): Machine,16,2 2.45 s, 16,4 2.60 s, 8,4 2.71 s.

`tasks.sieve.las.threads=gpu-auto, las -t gpu-auto`

Values: `gpu-auto` *Default:* not used

[CUDA build only] (for the GPU count; otherwise means `auto`). *Effect.* Each `las` chooses its own layout from its node: `Machine,`J`,2` with $J = \max(1, \lfloor \max(4, \lfloor c/g \rfloor)/2 \rfloor)$, where c is the number of CPUs in the process's affinity mask (`sched_getaffinity`, so `taskset` and Slurm limits count; else the hardware threads) and g the visible CUDA devices. Without a GPU, or with `CADO_LAS_GPU_SIEVE=0`: `auto`. In `cado-nfs.py` the rest of the layout (workunit limit, longer q-ranges) still applies. Examples: 32 CPUs, 1 GPU: `Machine,16,2`; under `taskset -c 0-7`: `Machine,4,2`.

Use. Clusters whose sieving nodes differ.

Code: `sieve/las-parallel.cpp:223-249`.

5.2 Other cado-nfs.py parameters

`tasks.compression`

Values: `auto`, `gz`, `zst` *Default:* `auto`

[any build]. *Effect.* Compression of the files the server writes (free relations, renumbering table, `dup1` output, purged file, merge history): `auto` uses `.zst` when `zstd` is installed on the server, else `.gz`. Files that clients write or read (sieving output, factor base) use `.zst` only with `zst`, since `auto` cannot know whether the clients' hosts have `zstd`. Another value raises an error. Ignored (no compression) when the task's `gzip` parameter is off. Measured: the six `dup2` files of a c120 read in 3.45 s with `gzip`, 0.35 s with `zstd`.

Code: `scripts/cadofactor/cadotask.py:1333-1360`.

`tasks.polyselect.batch`

Values: integer *Default:* the `polyselect_ropt` threads of a client (`tasks.polyselect.threads`)

[any build]. *Effect.* Polynomials per root optimization workunit. Upstream default: 5 rounded up to a multiple of the threads. With the new default, a c120's 48 polynomials (2-thread clients) make 24 workunits spread over all 16 clients; before, they went to 8 of the 16.

Code: `scripts/cadofactor/cadotask.py:2837-2842`.

`tasks.sqrt.threads`

Values: integer; `0` = choose *Default:* 0 (upstream: 1)

[any build]. *Effect.* Number of dependencies processed at once (`sqrt -t`). 0: `SqrtTask.auto_threads` chooses $\min(4, \lfloor A/2/\text{need} \rfloor, \text{CPUs})$, at least 1, with A the available memory (`MemAvailable`) and need = pairs × 1100 bytes × $\max(1, \text{bits}(N)/400)$. The step that writes the dependency files runs with 1. Measured (c120, GPU multiplication): one dependency 7.1 s, four at once 10.6 s; expected time to the factors 14.2 → 11.3 s.

Code: `scripts/cadofactor/toplevel.py:1098-1106`, `scripts/cadofactor/cadotask.py:6285-6309`, `scripts/cadofactor/cadotask.py:6346-6375`.

`tasks.filter.purge.keep`

Values: integer *Default:* from the parameter file; raised to 160 if lower

[any build]. *Effect.* For factoring (not discrete logarithms), a `keep` below 160 is raised to 160 with a log message: `bwc` over GF(2) needs more than $m + n$ rows (128 with $m = n = 64$) and `prep` loops forever otherwise. `params.qs.c40` now says 160 (was 32).

Code: `scripts/cadofactor/cadotask.py:4894-4902`.

`tasks.filter.dup2.threads, tasks.filter.threads`

Values: integer *Default:* $\min(6, \max(3, \lfloor \text{CPUs}/(2 \cdot \text{slices}) \rfloor))$

[any build]. *Effect.* `dup2` gets a `-t` option (threads computing roots mod p; `dup2`'s own default 3; `filter/dup2.cpp:248`), and `cado-nfs.py` runs the `dup2` slices in parallel. The task looks up `tasks.filter.dup2.threads` in the parameter hierarchy; only a value found under `tasks.filter` (`tasks.filter.dup2.threads` or `tasks.filter.threads`) is used. Otherwise, including when only `tasks.threads` is set, the default above applies: CPUs is `os.cpu_count()`, slices the number of `dup2` slices (6 on a 32-thread machine with two slices). Measured: a c60 with `-t 16` gave `dup2 -t 6`; with `tasks.filter.threads=5`, `-t 5`.

Code: `scripts/cadofactor/cadotask.py:4650-4664`, `scripts/cadofactor/cadoprograms.py:1221`.

5.3 bwc.pl parameters

These are `bwc.pl` parameters (`key=value` on its command line). `gpu=`, `gpu_block=` and `mpi_extra_args=` are not in the list of parameters that `cado-nfs.py` passes to `bwc.pl` (`BWC` in `scripts/cadofactor/cadoprograms.py:1354-1405`); under `cado-nfs.py` the defaults apply. `hosts=`, `hostfile=`, `mpi=`, `thr=`, `m=`, `n=`, `lingen_thr=` and `mm_impl=` are passed as `tasks.linalg.bwc.<name>`.

`gpu`

Values: `auto`, `1`, `0` *Default:* `auto`

Effect. `auto`: when `prime=2`, `mm_impl` is unset or `cuda`, and `cuda_devices` reports $g > 0$ GPUs (or `CADO_BWC_GPUS_PER_NODE` is set), then:

- `mm_impl=cuda`, one GPU per matrix block;
- several nodes (from `hosts=`, `hostfile=` or `SLURM_JOB_NUM_NODES`), `mpi` not given, MPI build: `mpi`=nodes×GPUs-per-node, `thr=1`;
- one node, $g > 1$, `mpi` not given, MPI build: `mpi`=g, `thr=1` (vectors on the GPUs, NCCL);
- otherwise `thr`=g without `mpi` (one thread per GPU, vectors on the host), or `thr=1` when `mpi` was given;
- `lingen_thr` becomes the `thr=` that was asked for, if it was left at its default `1x1`;
- `m=n=64` without `simd` becomes `m=n=simd=gpu_block`.

It prints a line "## GPU: ..." with the result. `0`: the CPU code. `1`: like `auto`, but stops if there is no GPU or no `cuda` backend.

Code: `linalg/bwc/bwc.pl:185-298`.

`gpu_block`

Values: integer *Default:* 128

Effect. With the GPU, the default `m=n=64` becomes `m=n=simd=gpu_block`. A 128-bit product costs the GPU about what a 64-bit one costs, so `krylov` and `mksol` need half as many. Not applied when the matrix has fewer than $16 \cdot 2 \cdot$`gpu_block` rows (read from the `.rw.bin` file when present). `64` keeps 64. Measured on the c140 matrix: 256 instead of 128 made the products 1.45 times as long and `lingen` nearly twice as long (total 231.9 vs 220.4 s), so 128 stays the default.

Code: `linalg/bwc/bwc.pl:204`, `linalg/bwc/bwc.pl:270-290`.

`hosts, hostfile (multi-node default)`

Values: comma-separated hosts / a file *Default:* none

Effect. Upstream parameters. With the GPU, the number of distinct hosts now also sets `mpi` (one job per GPU of every node, Open MPI with `--map-by node`, each job on the GPU of its local rank). An explicit `mpi=` is kept. Nodes need passwordless `ssh` between them. Via `cado-nfs.py`: `tasks.linalg.bwc.hosts=node1,node2` or `tasks.linalg.bwc.hostfile=`*file*. Tested only on one machine (`--show` output; a real multi-node run is still to be done).

Code: `linalg/bwc/bwc.pl:220-238`, `linalg/bwc/bwc.pl:253-257`.

6 Differences between the code and its documentation

The code was taken as correct. Commit `e80379d4b` fixed the differences found while writing this document: `las`'s GPU choice (one decision at start-up through the claim, the same local-rank variables for all its parts), `CADO_LAS_GPU_SIEVE` read the same way by `las`, `las -t gpu-auto` and `cado-nfs.py`, the `dup2` thread default, and the README files and comments (`las`'s claim, the `polyselect` device default, the CUDA architecture default in `linalg/bwc/README.gpu.md`, the subjob size, `mpi_extra_args`, `gpu=` and `gpu_block=` only when running `bwc.pl` directly, `ENABLE_CUDA` not settable as a `local.sh` variable).

A comment in `sieve/las-fill-gpu.cpp` that named a `CADO_LAS_GPU_FILL_CHECK` variable, which no code read, was corrected in the same series. No known difference remains.

7 Index

Name	Component	Default	Effect
`CADO_BWC_CUDA_DEVICE`	bwc	local rank	GPU of each matrix block
`CADO_BWC_CUDA_RESIDENT`	bwc	on	vectors stay on the GPU; `0`: on the host
`CADO_BWC_CUDA_VERBOSE`	bwc	unset	per-process product count and times
`CADO_BWC_GPU_COMM`	bwc	`nccl` if built	`nccl` or `mpi` transport between jobs
`CADO_BWC_GPUS_PER_NODE`	bwc.pl	local GPUs	GPUs per node for the multi-node layout
`CADO_BWC_SIMULATE_GPUS`	bwc (test)	unset	pretend n GPUs; also changes `cado-nfs.py`'s `las` layout
`CADO_CHARACTERS_GPU`	characters	on	`0`: CPU characters
`CADO_CHARACTERS_GPU_CHECK`	characters (test)	unset	recompute on CPU, abort on difference
`CADO_CHARACTERS_GPU_DEVICE`	characters	rank / claim	GPU of `characters`
`CADO_CLIENT_HOST`	client (test)	host name	host name sent to the server
`CADO_CLIENT_NGPU`	client	`nvidia-smi -L`	GPU count sent to the server
`CADO_CUDA_ARCHITECTURES`	build	see text	default `CMAKE_CUDA_ARCHITECTURES`
`CADO_FILTER_PARSE_THREADS`	filtering	4 (1 if < 8 threads)	relation parsing threads
`CADO_GPU_CLAIM_VERBOSE`	GPU claim	off	print each claim
`CADO_GPU_SIMULATE_NDEV`	GPU claim, sqrt (test)	unset	claim among n GPUs; n logical GPUs in `sqrt`
`CADO_LAS_GPU_BATCH`	las cofac.	65536	candidates per GPU batch
`CADO_LAS_GPU_COFAC`	las cofac.	on	`0`: CPU cofactorization
`CADO_LAS_GPU_COFILTER`	las cofac.	on	`0`: no GPU cofactor filter
`CADO_LAS_GPU_COFILTER_BITS`	las cofac. (test)	256/512 auto	`512` forces the wide kernel
`CADO_LAS_GPU_DEVICE`	las	rank / claim	GPU of `las` (set at start-up)
`CADO_LAS_GPU_FILL`	las sieving	on	`0`: CPU top-level bucket fill
`CADO_LAS_GPU_FILTER`	las cofac.	on	`0`: no strategy estimate filter on GPU
`CADO_LAS_GPU_FUSE`	las sieving	on	`0`: separate resieve/sieve passes
`CADO_LAS_GPU_NORMS_ROW`	las sieving	off	row-wise algebraic norms kernel
`CADO_LAS_GPU_OVERLAP`	las sieving	on	`0`: no CPU/GPU overlap in a subjob
`CADO_LAS_GPU_SIEVE`	las sieving	on	`0`: CPU siever
`CADO_LAS_GPU_SIEVE_CHECK`	las (test)	off	CPU sieves, GPU compared [slow]
`CADO_LAS_GPU_SIEVE_DEBUG`	las (test)	unset	print small sieve entries, mismatch details
`CADO_LAS_GPU_SIEVE_NORMS`	las (test)	unset	check the lognorms alone
`CADO_LAS_GPU_SLAB_LOG`	las sieving	L2 and factor base	slab of 2^k positions
`CADO_LAS_GPU_SPIN`	las sieving	off	spin instead of sleep on GPU waits

Name	Component	Default	Effect
CADO_LAS_GPU_WAIT	las cofac.	1.0 s	longest wait of a partial batch
CADO_LAS_NGPU	cado-nfs.py	server GPUs	GPUs per sieving node; 0: no GPU layout
CADO_NO_PIGZ	files	off	use gzip, not pigz
CADO_POLYSELECT_GPU	polyselect	on	0: all polyselect GPU code off
CADO_POLYSELECT_GPU_CHECK	polyselect (test)	unset	exact collision check [slow]
CADO_POLYSELECT_GPU_DEVICE	polyselect, ropt	rank / claim	GPU of polyselect and polyselect_ropt
CADO_POLYSELECT_GPU_SOPT	size opt.	FP64 ratio ≤ 4	0 CPU, 1 GPU
CADO_POLYSELECT_GPU_VERBOSE	polyselect	unset	counts at exit
CADO_ROPT_GPU	ropt	on	0: CPU root sieve and counts
CADO_ROPT_GPU_ALPHA	ropt	on	0: CPU root counts only
CADO_ROPT_GPU_CHECK	ropt (test)	unset	CPU recheck, abort on difference [slow]
CADO_ROPT_GPU_MIN	ropt	16384	shortest array sent to the GPU
CADO_ROPT_GPU_VERBOSE	ropt	unset	counts at exit
CADO_ROPT_SIEVE_BLOCK	ropt (test)	16384	CPU root sieve block size
CADO_SIEVE_WU_LIMIT	cado-nfs.py	set by layout	sieving workunits at a time (per host)
CADO_SQRT_GPU	sqrt	on	0: GMP products only
CADO_SQRT_GPU_CHECK	sqrt (test)	unset	compare with mpz_mul [slow]
CADO_SQRT_GPU_DEVICE	sqrt	all GPUs	restrict sqrt to one GPU
CADO_SQRT_GPU_MIN	sqrt	32768 limbs	smallest product on the GPU
CADO_SQRT_GPU_VERBOSE	sqrt	unset	counts at the end
CGBN_HOME	build	unset	where to find cgbn/cgbn.h
CMAKE_CUDA_ARCHITECTURES	build	see text	GPU code generated
CMAKE_CUDA_COMPILER, CUDACXX	build	nvcc on PATH	CUDA compiler
CUDA_VISIBLE_DEVICES	all GPU code	all	devices visible
ENABLE_CUDA	build	ON	build the GPU code
ENABLE_NCCL	build	ON	use NCCL in bwc
gpu (bwc.pl)	bwc.pl	auto	GPU defaults for bwc
gpu_block (bwc.pl)	bwc.pl	128	$m = n =$ simd with the GPU
hosts, hostfile (bwc.pl)	bwc.pl	none	nodes; one job per GPU of each
las -t gpu-auto	las	—	per-node Machine,J,2
MPI (configure)	build	unset	MPI build
MPI_LOCALRANKID	GPU selection	—	local rank (some programs)
NCCL_HOME, NCCL_INCLUDE_DIR, NCCL_LIBRARY	build	system paths	where NCCL is
NCCL_SOCKET_IFNAME	NCCL	NCCL's	network interface for NCCL
OMPI_COMM_WORLD_LOCAL_RANK	GPU selection	—	local rank
SLURM_JOB_NUM_NODES	bwc.pl	—	node count without hosts
SLURM_LOCALID	GPU selection	—	local rank
SOPT_TEST_NOCPU	sopt_gpu_test	unset	compare 2000 descents only
tasks.compression	cado-nfs.py	auto	.zst or .gz
tasks.filter.dup2.threads, tasks.filter.threads	cado-nfs.py	3 to 6, by CPUs	dup2 -t (only values under tasks.filter count)
tasks.filter.purge.keep	cado-nfs.py	≥ 160	raised for bwc
tasks.polyselect.batch	cado-nfs.py	client threads	polynomials per ropt workunit
tasks.sieve.las.threads=gpu-auto	cado-nfs.py	—	per-node layout
tasks.sqrt.threads	cado-nfs.py	0 (auto, ≤ 4)	dependencies at once
TMPDIR	GPU claim	/tmp	lock directory